

SEMANTIC CODE SEARCH

AI-POWERED RAG SYSTEM FOR CODEBASE UNDERSTANDING

Project Report

Retrieval-Augmented Generation over GitHub Repositories

Himani Jangid

B.Tech — Artificial Intelligence & Data Science

GitHub: <https://github.com/himani098/semantic-code-search>

Live Demo: <https://semantic-code-search-black.vercel.app>

Contents

Acknowledgement	2
1 Abstract	3
2 Introduction	3
3 Problem Statement	3
4 Objectives	4
5 Existing System & Limitations	4
6 Proposed System	5
7 System Architecture	5
8 Complete Project Workflow	6
9 Technologies Used	7
10 API Endpoints	7
11 Key Features	8
12 Frontend & Backend	8
12.1 Frontend	8
12.2 Backend	8
13 Deployment	9
14 Results	9
14.1 Application Screenshots	10
15 Advantages	10
16 Limitations	12
17 Future Scope	13
18 Conclusion	13
19 References	13
20 Architecture Overview	14

Acknowledgement

I would like to express my sincere gratitude to **GRRAS Solutions Pvt. Ltd.** for the opportunity to undergo hands-on training in Data Science and AI/ML, which laid the technical foundation for this project. The concepts covered during the training — including Python, SQL, machine learning, deep learning, NLP, and Generative AI with LLMs and RAG pipelines — were directly applied in building this system.

I would also like to thank my faculty guide for their continued guidance, valuable feedback, and encouragement throughout the development of this project, which helped shape it into a complete, deployable application rather than just a classroom exercise.

Finally, I am grateful to my institution for providing an environment that encourages practical, industry-relevant learning alongside academic coursework.

1 Abstract

Understanding an unfamiliar codebase is one of the most time-consuming parts of software development. Developers routinely rely on keyword-based tools such as `grep` or basic IDE search, which only match exact text and fail whenever the code and the question use different words for the same concept. This project, **Semantic Code Search**, addresses that gap by combining classical information retrieval with generative AI in a **Retrieval-Augmented Generation (RAG)** architecture. The system allows a user to index any public GitHub repository, after which they can ask natural-language questions about the codebase and receive accurate, citation-backed answers grounded in the actual source code. The implementation spans a complete stack — an AST-based code parser, a Sentence-Transformer embedding model, a ChromaDB vector store, a Google Gemini-powered answer generator, a FastAPI backend, and a React frontend — and is deployed live on Vercel and Railway. An automated evaluation framework was also built to objectively measure answer quality, giving a transparent, measured account of the system’s current performance (60% accuracy on a benchmark question set) rather than an unverified claim of correctness.

2 Introduction

Software teams today work with codebases that can span hundreds of thousands of lines across hundreds of files. New developers joining such a project, or even experienced developers moving into an unfamiliar part of the code, routinely need to answer questions like “*where is user authentication handled?*” or “*how are API responses formatted?*” Traditionally, answering such questions means manually searching for keywords and reading through multiple files, hoping to guess the right terminology the original author used.

This project applies a modern AI technique — Retrieval-Augmented Generation — to this everyday developer problem. Rather than only matching keywords, the system understands the **meaning** of a question and the **meaning** of each function or class in the codebase, retrieves the most relevant pieces of code based on that meaning, and then uses a large language model to compose a clear, grounded, and cited answer.

3 Problem Statement

Existing code search tools are limited to **lexical (keyword) matching** — they can only find code that contains the exact words typed into the search box. This creates two major problems:

1. **Terminology mismatch:** A developer asking “how is login handled?” will get no results if the code implements this using a function named `verify_user` or `authenticate`, since no exact keyword overlap exists.

2. **No synthesis:** Even when relevant files are found, the developer must manually read and piece together the logic themselves — the tool offers no explanation, summary, or answer, only a list of matching lines.

There is a need for a tool that understands the **semantic intent** behind a question and can generate a direct, explained answer grounded in the actual code, rather than just a list of text matches.

4 Objectives

- Automatically parse and understand the structure of a Python codebase using AST-based analysis.
- Convert code into a searchable semantic (vector) representation using embeddings.
- Retrieve the most relevant code for a given natural-language question using vector similarity search.
- Generate accurate, citation-backed answers using a Large Language Model, grounded strictly in retrieved code.
- Provide a usable, deployed web interface for indexing repositories and asking questions.
- Track system performance and answer quality through structured logging, a user feedback mechanism, and automated evaluation.

In one line: Paste a GitHub repo URL, ask a plain-English question about the code, and get a grounded answer with the exact file and function it came from.

5 Existing System & Limitations

Currently available approaches to code search fall into a few categories:

Existing Approach	How it Works	Key Limitations
Text editor / IDE search (Ctrl+F)	Exact string matching within open files	No cross-file understanding; requires knowing exact terms used
grep / command-line search	Regex or keyword matching across files	Purely lexical; no understanding of code meaning or structure; results are unranked raw text matches

Existing Approach	How it Works	Key Limitations
GitHub’s built-in code search	Indexed keyword/text search across a repository	Still keyword-based; cannot answer a question in natural language or synthesize an explanation
Reading documentation	Manually written explanations of the codebase	Often outdated, incomplete, or missing entirely for internal/legacy code

Common limitation across all of these: none of them understand *semantic meaning* — they treat code as plain text rather than as a structured, meaningful artifact — and none of them can generate a direct, explained, natural-language answer to a question.

6 Proposed System

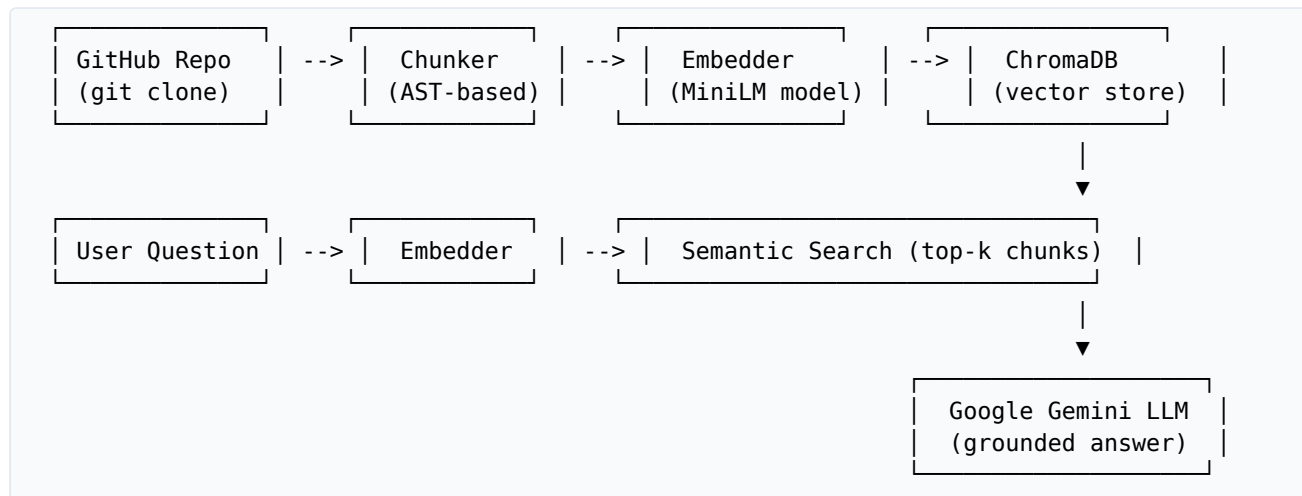
Semantic Code Search replaces keyword matching with a full Retrieval-Augmented Generation pipeline:

- Code is parsed into logically meaningful units (functions and classes) rather than arbitrary text blocks.
- Each unit is converted into a vector embedding that captures its semantic meaning.
- A user’s natural-language question is embedded the same way, and the system retrieves the most semantically similar code — regardless of exact keyword overlap.
- A Large Language Model (Google Gemini) is then used to read the retrieved code and generate a direct answer in plain English, explicitly citing the file and function it came from, and explicitly stating when it cannot find an answer rather than guessing.

This directly solves both limitations identified above: it understands meaning rather than exact text, and it produces a synthesized, explained answer rather than a raw list of matches.

7 System Architecture

The application follows a **7-step RAG pipeline**, combining an offline indexing phase with an online query phase:



Indexing phase (offline, triggered once per repository): GitHub Repo → Chunker → Embedder → ChromaDB

Query phase (online, triggered per question): User Question → Embedder → Semantic Search → Gemini LLM → Answer

8 Complete Project Workflow

The end-to-end workflow, from the moment a user submits a repository to the moment they receive an answer, proceeds as follows:

1. **Repository submission:** The user pastes a public GitHub repository URL into the frontend and clicks “Index Repository.”
2. **Cloning:** The backend clones the repository into a temporary directory using Git-Python.
3. **Change detection:** A SHA-256 hash is computed for every file and compared against previously stored hashes, to detect whether the repository content has actually changed since a prior indexing run.
4. **Chunking:** Every Python file is parsed using the ast module, and each function and class is extracted as an individual, logically complete chunk — along with its file path and starting line number.
5. **Embedding generation:** Each code chunk is converted into a vector embedding using the all-MiniLM-L6-v2 Sentence Transformer model, running locally at no API cost.
6. **Vector storage:** The embeddings, source code, and metadata (file, function name, line number) are stored in a persistent ChromaDB collection, ready to be searched.
7. **Question submission:** The user types a natural-language question (e.g. “How is user authentication handled?”) and submits it.
8. **Question embedding:** The question is converted into a vector using the same embedding model, ensuring it lives in the same semantic space as the indexed code.
9. **Semantic retrieval:** ChromaDB performs a similarity search and returns the top-k (5) most semantically relevant code chunks.

10. **Answer generation:** The question and retrieved chunks are formatted into a structured prompt and sent to Google Gemini (gemini-2.5-flash), which is instructed to answer only from the provided code, cite file/function names, and explicitly say when it cannot find an answer.
11. **Response delivery:** The generated answer, along with the underlying source chunks, is returned to the frontend and displayed to the user for verification.
12. **Logging and feedback:** The query, response time, and chunk count are logged; the user may optionally rate the answer as helpful or not, which is stored for future analysis.

9 Technologies Used

Layer	Technology
Frontend	React, Vite, Axios
Backend	Python, FastAPI
Code Parsing	Python ast module
Embeddings	Sentence Transformers (all-MiniLM-L6-v2)
Vector Database	ChromaDB
LLM	Google Gemini (gemini-2.5-flash)
Repository Handling	GitPython
Feedback / Logging Storage	SQLite, flat-file logs
Containerization	Docker
Deployment (Frontend)	Vercel
Deployment (Backend)	Railway

10 API Endpoints

The backend exposes a REST API built with FastAPI:

Method	Endpoint	Description
POST	/index	Clones and indexes a GitHub repository
POST	/query	Accepts a natural-language question, returns an answer with sources

Method	Endpoint	Description
POST	/feedback	Stores a user's rating (helpful / not helpful) of a given answer
GET	/	Health check

CORS is enabled on the backend to allow the deployed frontend (on a different origin) to communicate with it.

11 Key Features

- **Semantic code search** using Sentence Transformer embeddings, rather than keyword matching.
 - **AI-generated, grounded answers** using Google Gemini, with explicit file/function citations.
 - **One-click GitHub repository indexing** — no manual setup required by the user.
 - **ChromaDB vector database** for fast, persistent semantic retrieval.
 - **Query logging** for observability into response time and retrieval size.
 - **User feedback collection** (helpful / not helpful) on every generated answer.
 - **Automated evaluation framework** to objectively measure answer accuracy.
 - **Smart re-indexing awareness** using SHA-256 file hashing to detect actual content changes.
 - **REST API** built with FastAPI, cleanly separating backend logic from the frontend.
 - **Interactive React frontend** with a clear two-step flow: index, then ask.
-

12 Frontend & Backend

12.1 Frontend

Built with **React** and **Vite**, the frontend provides a minimal, focused interface: a field to submit a GitHub repository URL for indexing, and a search box to ask questions once indexing is complete. Answers are displayed together with the exact source code snippets used to generate them, allowing the user to visually verify the AI's response against the real code. **Axios** is used to communicate with the backend REST API.

12.2 Backend

Built with **FastAPI**, the backend orchestrates the entire RAG pipeline: repository cloning, AST-based chunking, embedding generation, vector storage and retrieval, and LLM-based answer generation. It exposes a small, clean set of REST endpoints (see *API Endpoints*), handles CORS for cross-origin requests from the deployed frontend, and

persists feedback and query logs for later analysis. The backend is packaged with a Dockerfile for consistent, reproducible deployment.

13 Deployment

Component	Platform	URL
Frontend	Vercel	https://semantic-code-search-black.vercel.app
Backend	Railway	https://semantic-code-search-production-d7e1.up.railway.app

The backend is containerized using Docker, and both services are connected to the project's GitHub repository for straightforward, repeatable deployment.

Live and working: both frontend and backend are deployed and publicly reachable, not just running locally.

14 Results

To objectively measure system performance rather than relying on informal impressions, an automated evaluation script (`evaluate.py`) was run against a fixed set of test questions on the deployed `/query` endpoint, checking whether an expected keyword appeared in the generated answer.

Sample evaluation results (tested against the Flask repository):

Question	Expected Key-word	Result
What does <code>url_for</code> do?	<code>url_for</code>	Passed
How are templates rendered?	<code>template</code>	Passed
How does Flask handle routing?	<code>routing</code>	Failed
How are requests parsed?	<code>request</code>	Failed
How are responses created?	<code>response</code>	Passed

Measured Accuracy: 3/5 (60%) on a fixed test set of natural-language questions against the indexed Flask repository.

The failures both correspond to broad, conceptual questions (e.g. “how does routing work”) that logically span multiple functions rather than matching a single clearly-relevant chunk — a concrete, honestly-reported limitation discussed further below.

14.1 Application Screenshots

Home Page — the main interface for submitting a repository and asking a question.

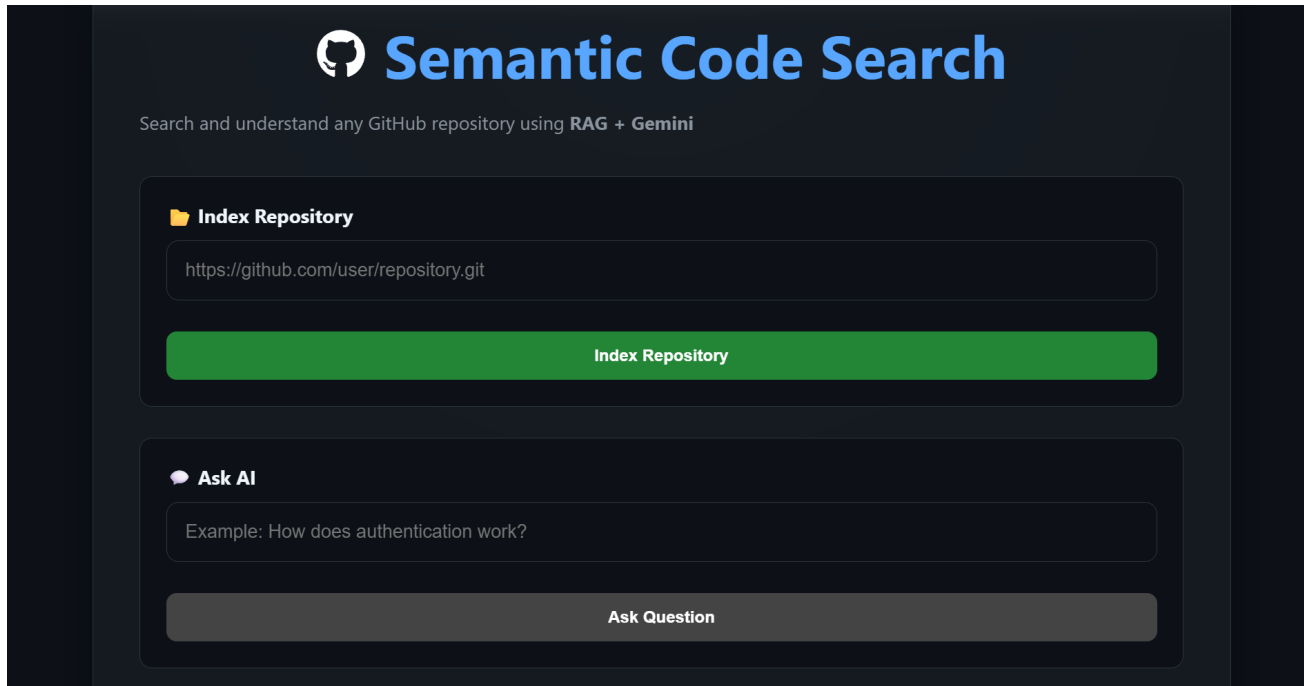


Figure 1: Home Page

Repository Indexed — confirmation shown once cloning, chunking, embedding, and storage complete.

AI-Generated Response — an answer generated by Gemini, grounded in retrieved code chunks, with a feedback option.

Source Files — the actual retrieved code snippets used to generate the answer above.

15 Advantages

- Understands the **meaning** of a question, not just its exact keywords — solving the core limitation of existing tools.
- Produces a **direct, explained answer** rather than a raw list of file matches, saving significant developer time.
- **Grounded and citation-backed:** answers reference the specific file and function they came from, allowing easy verification and reducing blind trust in AI output.
- **Low operating cost:** embedding generation runs locally with no per-query API cost; only the final answer generation calls an external LLM.

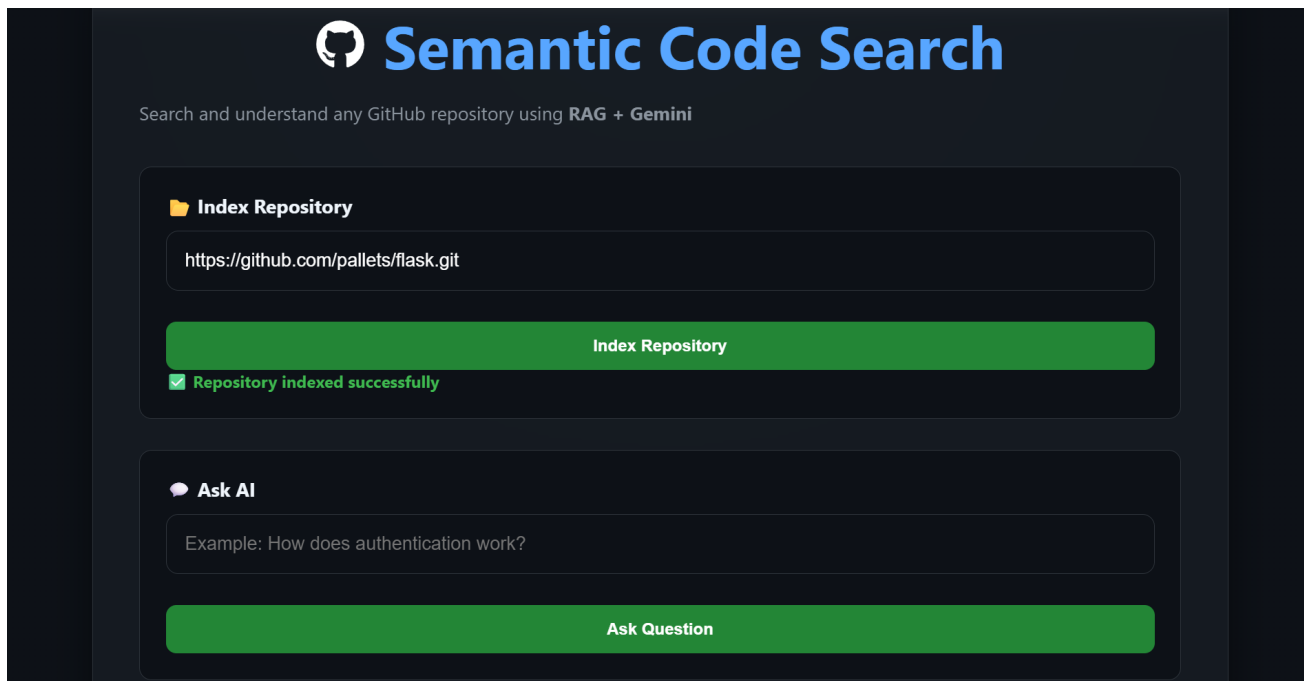


Figure 2: Repository Indexed

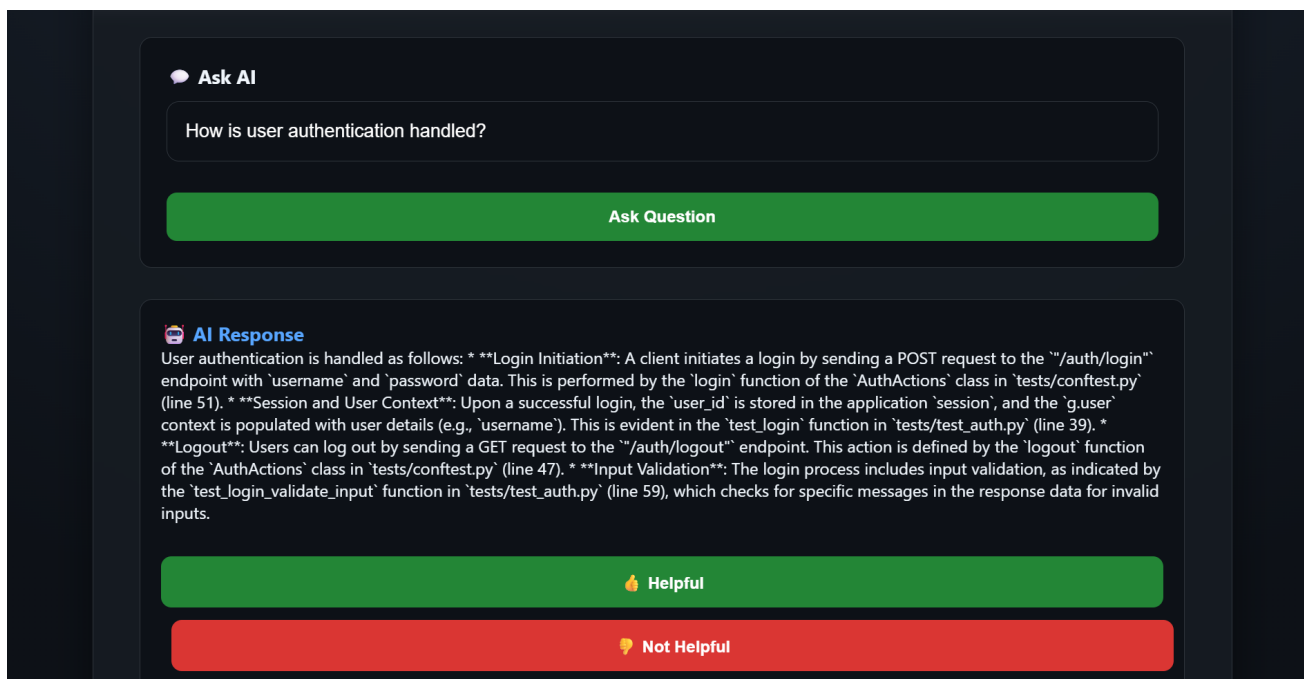


Figure 3: AI Response

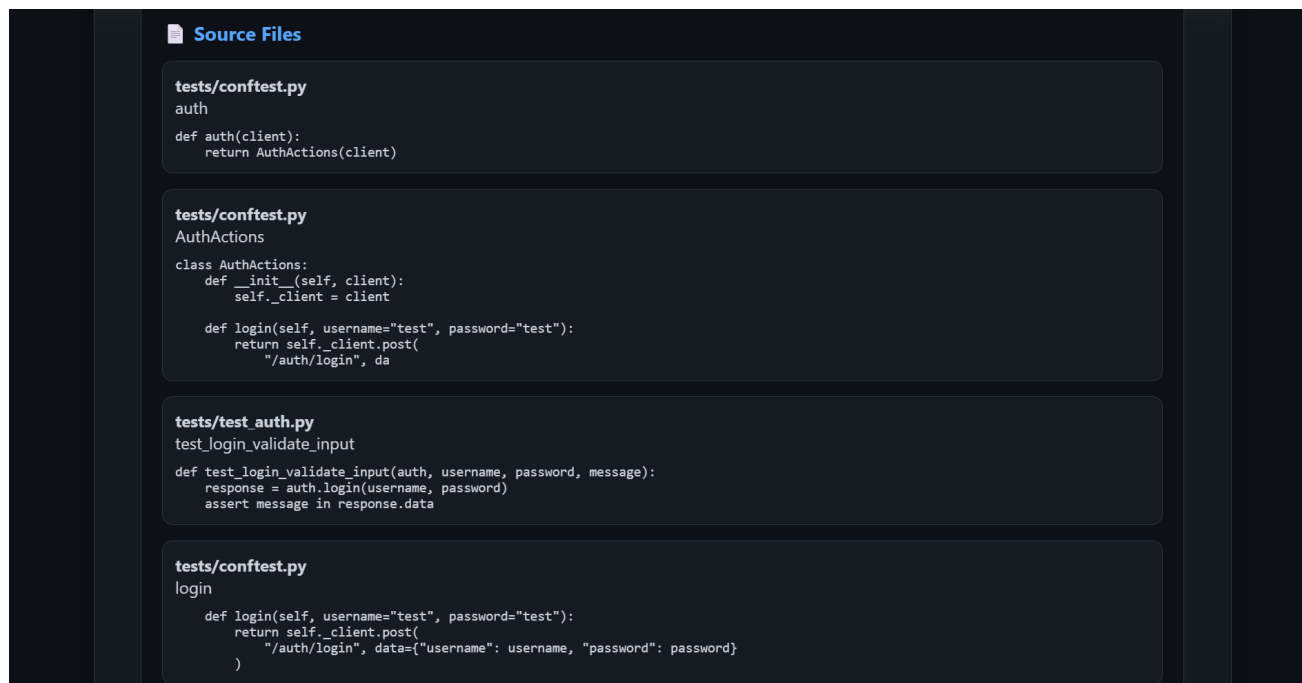


Figure 4: Source Files

- **Fully deployed, end-to-end system** — not just a local script or notebook — demonstrating real-world software engineering practices (REST API, containerization, CI-friendly deployment).
- **Built-in accountability:** logging, user feedback, and automated evaluation are part of the system itself, not an afterthought.

16 Limitations

- Currently supports **Python codebases only**; other languages are not parsed or chunked.
- Indexing is capped at **200 chunks per repository** to keep response times reasonable on the deployed infrastructure, which may leave parts of very large repositories unindexed.
- Only **one repository can be indexed at a time**; indexing a new repository replaces the existing index.
- Measured answer accuracy is **60%** on the benchmark test set — broad, conceptual questions that span multiple functions are harder for the system to answer correctly than narrow, specific ones.
- **No user authentication** — the system does not currently support multiple users with separate indexed repositories or histories.
- Re-indexing currently detects that a repository *has* changed via file hashing, but still reprocesses the entire repository rather than only the changed files.

17 Future Scope

- **Multi-language support** — extend chunking beyond Python using tree-sitter, to support JavaScript, Java, C++, and other languages.
 - **Multiple repository support** — allow indexing and querying across more than one repository simultaneously.
 - **Improved retrieval accuracy** — address the gaps identified in evaluation through better chunk ranking, hybrid keyword + semantic search, or expanded top-k retrieval for broader conceptual questions.
 - **User authentication** — support multiple users, each with their own indexed repositories and query history.
 - **Advanced analytics dashboard** — visualize query logs and feedback trends over time to track system health and usage patterns.
 - **True incremental indexing** — extend the existing hash-based change detection so that only changed files are re-chunked and re-embedded, rather than the full repository.
-

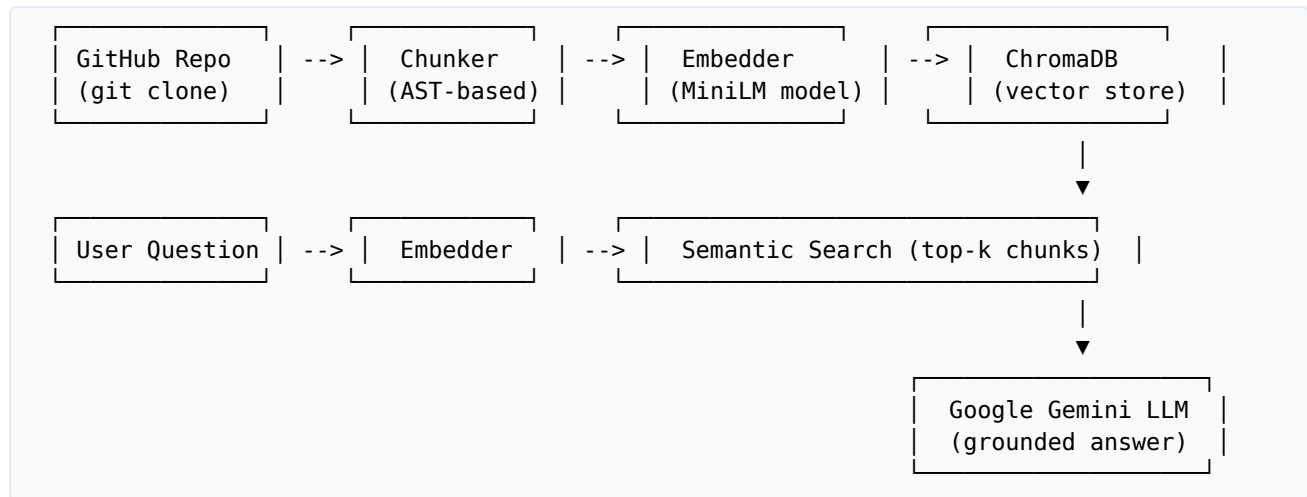
18 Conclusion

This project demonstrates a complete, end-to-end Retrieval-Augmented Generation application — from data ingestion (repository cloning and AST-based code parsing), through the ML core (embeddings and vector search), to a generative AI layer (Google Gemini) that produces grounded, cited answers. Beyond the core ML pipeline, the project incorporates practical software engineering concerns often missing from student projects: a real REST API, a deployed full-stack web application, structured logging, a user feedback mechanism, and a measured, honestly-reported evaluation of system accuracy. Together, these demonstrate not just the ability to train or call a model, but to build, evaluate, and ship a usable AI-powered product that solves a real, everyday developer problem.

19 References

1. Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS).
 2. Reimers, N. and Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of EMNLP.
 3. ChromaDB Documentation. <https://docs.trychroma.com>
 4. FastAPI Documentation. <https://fastapi.tiangolo.com>
 5. Google Gemini API Documentation. <https://ai.google.dev/gemini-api/docs>
 6. Sentence Transformers Documentation. <https://www.sbert.net>
 7. React Documentation. <https://react.dev>
 8. Project Repository: <https://github.com/himani098/semantic-code-search>
 9. Live Application: <https://semantic-code-search-black.vercel.app>
-

20 Architecture Overview



Summary: Two phases power the system — an offline **indexing pipeline** (repo → chunks → embeddings → vector store) and an online **query pipeline** (question → embedding → semantic search → LLM → grounded answer).